Document number: LEWG, EWG, SG14, SG6: P0037R0
Date: 2015-09-28
Project: Programming Language C++, Library Evolution WG, SG14
Reply-to: John McFarlane, fixed-point@john.mcfarlane.name

Fixed-Point Real Numbers

I. Introduction

This proposal introduces a system for performing binary fixed-point arithmetic using built-in integral types.

II. Motivation

Floating-point types are an exceedingly versatile and widely supported method of expressing real numbers on modern architectures.

However, there are certain situations where fixed-point arithmetic is preferable. Some systems lack native floating-point registers and must emulate them in software. Many others are capable of performing some or all operations more efficiently using integer arithmetic. Certain applications can suffer from the variability in precision which comes from a dynamic radix point [1]. In situations where a variable exponent is not desired, it takes valuable space away from the significand and reduces precision.

Built-in integer types provide the basis for an efficient representation of binary fixed-point real numbers. However, laborious, error-prone steps are required to normalize the results of certain operations and to convert to and from fixed-point types.

A set of tools for defining and manipulating fixed-point types is proposed. These tools are designed to make work easier for those who traditionally use integers to perform low-level, high-performance fixed-point computation.

III. Impact On the Standard

This proposal is a pure library extension. It does not require changes to any standard classes, functions or headers.

IV. Design Decisions

The design is driven by the following aims in roughly descending order:

1. to automate the task of using integer types to perform low-level binary fixed-point arithmetic;
2. to facilitate a style of code that is intuitive to anyone who is comfortable with integer and floating-point arithmetic;
3. to avoid type promotion, implicit conversion or other behavior that might lead to surprising results and
4. to preserve significant digits at the expense of insignificant digits, i.e. to prefer underflow to overflow.

Class Template

Fixed-point numbers are specializations of

```
template <class ReprType, int Exponent>
class fixed_point;
```

where the template parameters are described as follows.

Â`ReprType`Â Type Template Parameter

This parameter identifies the capacity and signedness of the underlying type used to represent the value. In other words, the size of the resulting type will be Â`sizeof(ReprType)`Â and it will be signed iff Â`is_signed<ReprType>::value`Â is true. The default is Â`int`Â.

Â`ReprType`Â must be a fundamental integral type and should not be the largest size. Suitable types include: Â`std::int8_t`Â,

Â`std::uint8_tÂ`, Â`std::int16_tÂ`, Â`std::uint16_tÂ`, Â`std::int32_tÂ` and Â`std::uint32_tÂ`. In limited situations, Â`std::int64_tÂ` and Â`std::uint64_tÂ` can be used. The reasons for these limitations relate to the difficulty in finding a type that is suitable for performing lossless integer multiplication.

Â`ExponentÂ` Non-Type Template Parameter

The exponent of a fixed-point type is the equivalent of the exponent field in a floating-point type and shifts the stored value by the requisite number of bits necessary to produce the desired range. The default value of Â`ExponentÂ` is zero, giving Â`fixed_point<T>Â` the same range as Â`TÂ`.

The resolution of a specialization of Â`fixed_pointÂ` is

```
pow(2, Exponent)
```

and the minimum and maximum values are

```
std::numeric_limits<ReprType>::min() * pow(2, Exponent)
```

and

```
std::numeric_limits<ReprType>::max() * pow(2, Exponent)
```

respectively.

Any usage that results in values of Â`ExponentÂ` which lie outside the range, (Â`INT_MIN / 2Â`, Â`INT_MAX / 2Â`), may result in undefined behavior and/or overflow or underflow. This range of exponent values is far in excess of the largest built-in floting-point type and should be adequate for all intents and purposes.

Â`make_fixedÂ` and Â`make_ufixedÂ` Helper Type

The Â`ExponentÂ` template parameter is versatile and concise. It is an intuitive scale to use when considering the full range of positive and negative exponents a fixed-point type might possess. It also corresponds to the exponent field of built-in floating-point types.

However, most fixed-point formats can be described more intuitively by the cardinal number of integer and/or fractional digits they contain. Most users will prefer to distinguish fixed-point types using these parameters.

For this reason, two aliases are defined in the style of Â`make_signedÂ`.

These aliases are declared as:

```
template <unsigned IntegerDigits, unsigned FractionalDigits = 0, bool IsSigned = true>
  using make_fixed;
```

and

```
template <unsigned IntegerDigits, unsigned FractionalDigits = 0>
using make_ufixed;
```

They resolve to a Â`fixed_pointÂ` specialization with the given signedness and number of integer and fractional digits. They may contain additional integer and fractional digits.

For example, one could define and initialize an 8-bit, unsigned, fixed-point variable with four integer digits and four fractional digits:

```
make_ufixed<4, 4> value { 15.9375 };
```

or a 32-bit, signed, fixed-point number with two integer digits and 29 fractional digits:

```
make_fixed<2, 29> value { 3.141592653 };
```

Conversion

Fixed-point numbers can be explicitly converted to and from built-in arithmetic types.

While effort is made to ensure that significant digits are not lost during conversion, no effort is made to avoid rounding errors. Whatever would happen when converting to and from an integer type largely applies to `fixed_point` objects also. For example:

```
make_ufixed<4, 4>(.006) == make_ufixed<4, 4>(0)
```

...equates to `true` and is considered a acceptable rounding error.

Operator Overloads

Any operators that might be applied to integer types can also be applied to fixed-point types. A guiding principle of operator overloads is that they perform as little run-time computation as is practically possible.

With the exception of shift and comparison operators, binary operators can take any combination of:

- one or two fixed-point arguments and
- zero or one arguments of any arithmetic type, i.e. a type for which `is_arithmetic` is true.

Where the inputs are not identical fixed-point types, a simple set of promotion-like rules are applied to determine the return type:

1. If both arguments are fixed-point, a type is chosen which is the size of the larger type, is signed if either input is signed and has the maximum integer bits of the two inputs, i.e. cannot lose high-significance bits through conversion alone.
2. If one of the arguments is a floating-point type, then the type of the result is the smallest floating-point type of equal or greater size than the inputs.
3. If one of the arguments is an integral type, then the result is the other, fixed-point type.

Some examples:

```
make_ufixed<5, 3>{8} + make_ufixed<4, 4>{3} == make_ufixed<5, 3>{11};
make_ufixed<5, 3>{8} + 3 == make_ufixed<5, 3>{11};
make_ufixed<5, 3>{8} + float{3} == float{11};
```

The reasoning behind this choice is a combination of predictability and performance. It is explained for each rule as follows:

1. ensures that the least computation is performed where fixed-point types are used exclusively. Aside from multiplication and division requiring shift operations, should require similar computational costs to equivalent integer operations;
2. loosely follows the promotion rules for mixed-mode arithmetic, ensures values with exponents far beyond the range of the fixed-point type are catered for and avoids costly conversion from floating-point to integer and
3. preserves the input fixed-point type whose range is far more likely to be of deliberate importance to the operation.

Shift operator overloads require an integer type as the right-hand parameter and return a type which is adjusted to accommodate the new value without risk of overflow or underflow.

Comparison operators convert the inputs to a common result type following the rules above before performing a comparison and returning `true` or `false`.

Overflow

Because arithmetic operators return a result of equal capacity to their inputs, they carry a risk of overflow. For instance,

```
make_fixed<4, 3>(15) + make_fixed<4, 3>(1)
```

causes overflow because because a type with 4 integer bits cannot store a value of 16.

Overflow of any bits in a signed or unsigned fixed-point type is classed as undefined behavior. This is a minor deviation from built-in integer arithmetic where only signed overflow results in undefined behavior.

Underflow

The other typical cause of lost bits is underflow where, for example,

```
make_fixed<7, 0>(15) / make_fixed<7, 0>(2)
```

results in a value of 7. This results in loss of precision but is generally considered acceptable.

However, when all bits are lost due to underflow, the value is said to be flushed and this is classed as undefined behavior.

Dealing With Overflow and Flushes

Errors resulting from overflow and flushes are two of the biggest headaches related to fixed-point arithmetic. Integers suffer the same kinds of errors but are somewhat easier to reason about as they lack fractional digits. Floating-point numbers are largely shielded from these errors by their variable exponent and implicit bit.

Three strategies for avoiding overflow in fixed-point types are presented:

1. simply leave it to the user to avoid overflow;
2. promote the result to a larger type to ensure sufficient capacity or
3. adjust the exponent of the result upward to ensure that the top limit of the type is sufficient to preserve the most significant digits at the expense of the less significant digits.

For arithmetic operators, choice 1) is taken because it most closely follows the behavior of integer types. Thus it should cause the least surprise to the fewest users. This makes it far easier to reason about in code where functions are written with a particular type in mind. It also requires the least computation in most cases.

Choices 2) and 3) are more robust to overflow events. However, they represent different trade-offs and neither one is the best fit in all situations. For these reasons, they are presented as named functions.

Type Promotion

Function template, `Â promoteÂ`, borrows a term from the language feature which avoids integer overflow prior to certain operations. It takes a `Â fixed_pointÂ` object and returns the same value represented by a larger `Â fixed_pointÂ` specialization.

For example,

```
promote(make_fixed<5, 2>(15.5))
```

is equivalent to

```
make_fixed<11, 4>(15.5)
```

Complimentary function template, `Â demoteÂ`, reverses the process, returning a value of a smaller type.

Named Arithmetic Functions

The following named function templates can be used as a hassle-free alternative to arithmetic operators in situations where the aim is to avoid overflow.

Unary functions:

```
trunc_reciprocal, trunc_square, trunc_sqrt,
promote_reciprocal, promote_square
```

Binary functions:

```
trunc_add, trunc_subtract, trunc_multiply, trunc_divide
trunc_shift_left, trunc_shift_right,
promote_add, promote_sub, promote_multiply, promote_divide
```

Some notes:

1. The `trunc_` functions return the result as a type no larger than the inputs and with an exponent adjusted to avoid overflow;
2. the `promote_` functions return the result as a type large enough to avoid overflow and underflow;
3. the `_multiply` and `_square` functions are not guaranteed to be available for 64-bit types;
4. the `_multiply` and `_square` functions produce undefined behavior when all input parameters are the *most negative number*;
5. the `_square` functions return an unsigned type;
6. the `_add`, `_subtract`, `_multiply` and `_divide` functions take heterogeneous `fixed_point` specializations;
7. the `_divide` and `_reciprocal` functions in no way guard against divide-by-zero errors;
8. the `trunc_shift_` functions return results of the same type as their first input parameter and
9. the list is by no means complete.

Example

The following example calculates the magnitude of a 3-dimensional vector.

```
template <class Fp>
constexpr auto magnitude(const Fp & x, const Fp & y, const Fp & z)
-> decltype(trunc_sqrt(trunc_add(trunc_square(x), trunc_square(y), trunc_square(z))))
{
    return trunc_sqrt(trunc_add(trunc_square(x), trunc_square(y), trunc_square(z)));
}
```

Calling the above function as follows

```
static_cast<double>(magnitude(
    make_ufixed<4, 12>(1),
    make_ufixed<4, 12>(4),
    make_ufixed<4, 12>(9)));
```

returns the value, 9.890625.

V. Technical Specification

Header \ Synopsis

```
namespace std {
  template <class ReprType, int Exponent> class fixed_point;

  template <unsigned IntegerDigits, unsigned FractionalDigits = 0, bool IsSigned = true>
    using make_fixed;
  template <unsigned IntegerDigits, unsigned FractionalDigits = 0>
    using make_ufixed;
  template <class ReprType, int IntegerDigits>
    using make_fixed_from_repr;

  template <class FixedPoint>
    using promote_result;
  template <class FixedPoint>
    promote_result<FixedPoint>
      constexpr promote(const FixedPoint & from) noexcept;

  template <class FixedPoint>
    using demote_result;
  template <class FixedPoint>
    demote_result<FixedPoint>
      constexpr demote(const FixedPoint & from) noexcept;

  template <class ReprType, int Exponent>
      constexpr bool operator==(
        const fixed_point<ReprType, Exponent> & lhs,
        const fixed_point<ReprType, Exponent> & rhs) noexcept;
  template <class ReprType, int Exponent>
    constexpr bool operator!=(
```

```
      const fixed_point<ReprType, Exponent> & lhs,
    const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    constexpr bool operator<(
      const fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    constexpr bool operator>(
      const fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    constexpr bool operator>=(
      const fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    constexpr bool operator<=(
      const fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;

template <class ReprType, int Exponent>
    constexpr fixed_point<ReprType, Exponent> operator-(
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    constexpr fixed_point<ReprType, Exponent> operator+(
      const fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    constexpr fixed_point<ReprType, Exponent> operator-(
      const fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    fixed_point<ReprType, Exponent> & operator+=(
      fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    fixed_point<ReprType, Exponent> & operator-=(
      fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    fixed_point<ReprType, Exponent> & operator*=(
      fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;
template <class ReprType, int Exponent>
    fixed_point<ReprType, Exponent> & operator/=(
      fixed_point<ReprType, Exponent> & lhs,
      const fixed_point<ReprType, Exponent> & rhs) noexcept;

template <class Lhs, class Rhs>
    constexpr auto operator==(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
    constexpr auto operator!=(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
    constexpr auto operator<(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
    constexpr auto operator>(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
    constexpr auto operator>=(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
    constexpr auto operator<=(const Lhs & lhs, const Rhs & rhs) noexcept;

template <class Lhs, class Rhs>
    constexpr auto operator+(
      const Lhs & lhs,
      const Rhs & rhs) noexcept;
template <class Lhs, class Rhs>
    constexpr auto operator-(
      const Lhs & lhs,
      const Rhs & rhs) noexcept;
template <class LhsReprType, int LhsExponent, class RhsReprType, int RhsExponent>
    constexpr auto operator*(
      const fixed_point<LhsReprType, LhsExponent> & lhs,
      const fixed_point<RhsReprType, RhsExponent> & rhs) noexcept;
template <class LhsReprType, int LhsExponent, class RhsReprType, int RhsExponent>
    constexpr auto operator/(
      const fixed_point<LhsReprType, LhsExponent> & lhs,
      const fixed_point<RhsReprType, RhsExponent> & rhs) noexcept;
template <class LhsReprType, int LhsExponent, class Integer>
    constexpr auto operator*(
      const fixed_point<LhsReprType, LhsExponent> & lhs,
```

```
      const Integer & rhs) noexcept;
template <class LhsReprType, int LhsExponent, class Integer>
    constexpr auto operator/(
      const fixed_point<LhsReprType, LhsExponent> & lhs,
      const Integer & rhs) noexcept;
template <class Integer, class RhsReprType, int RhsExponent>
    constexpr auto operator*(
      const Integer & lhs,
      const fixed_point<RhsReprType, RhsExponent> & rhs) noexcept;
template <class Integer, class RhsReprType, int RhsExponent>
    constexpr auto operator/(
      const Integer & lhs,
      const fixed_point<RhsReprType, RhsExponent> & rhs) noexcept;
template <class LhsReprType, int LhsExponent, class Float>
    constexpr auto operator*(
      const fixed_point<LhsReprType, LhsExponent> & lhs,
      const Float & rhs) noexcept;
template <class LhsReprType, int LhsExponent, class Float>
    constexpr auto operator/(
      const fixed_point<LhsReprType, LhsExponent> & lhs,
      const Float & rhs) noexcept;
template <class Float, class RhsReprType, int RhsExponent>
    constexpr auto operator*(
      const Float & lhs,
      const fixed_point<RhsReprType, RhsExponent> & rhs) noexcept;
template <class Float, class RhsReprType, int RhsExponent>
    constexpr auto operator/(
      const Float & lhs,
      const fixed_point<RhsReprType, RhsExponent> & rhs) noexcept;
template <class LhsReprType, int Exponent, class Rhs>
    fixed_point<LhsReprType, Exponent> & operator+=(fixed_point<LhsReprType, Exponent> & lhs, const Rhs & rhs) noexce
template <class LhsReprType, int Exponent, class Rhs>
    fixed_point<LhsReprType, Exponent> & operator-=(fixed_point<LhsReprType, Exponent> & lhs, const Rhs & rhs) noexce
template <class LhsReprType, int Exponent>
template <class Rhs, typename std::enable_if<std::is_arithmetic<Rhs>::value, int>::type Dummy>
    fixed_point<LhsReprType, Exponent> &
    fixed_point<LhsReprType, Exponent>::operator*=(const Rhs & rhs) noexcept;
template <class LhsReprType, int Exponent>
template <class Rhs, typename std::enable_if<std::is_arithmetic<Rhs>::value, int>::type Dummy>
    fixed_point<LhsReprType, Exponent> &
    fixed_point<LhsReprType, Exponent>::operator/=(const Rhs & rhs) noexcept;
template <class ReprType, int Exponent>
    constexpr fixed_point<ReprType, Exponent>
      sqrt(const fixed_point<ReprType, Exponent> & x) noexcept;
template <class FixedPoint, unsigned N = 2>
    using trunc_add_result;

template <class FixedPoint, class ... Tail>
    trunc_add_result<FixedPoint, sizeof...(Tail) + 1>
    constexpr trunc_add(const FixedPoint & addend1, const Tail & ... addend_tail);
template <class Lhs, class Rhs = Lhs>
    using trunc_subtract_result;
template <class Lhs, class Rhs>
    trunc_subtract_result<Lhs, Rhs>
      constexpr trunc_subtract(const Lhs & minuend, const Rhs & subtrahend);
template <class Lhs, class Rhs = Lhs>
    using trunc_multiply_result;

template <class Lhs, class Rhs>
    trunc_multiply_result<Lhs, Rhs>
      constexpr trunc_multiply(const Lhs & lhs, const Rhs & rhs) noexcept;
template <class FixedPointDividend, class FixedPointDivisor = FixedPointDividend>
    using trunc_divide_result;
template <class FixedPointDividend, class FixedPointDivisor>
    trunc_divide_result<FixedPointDividend, FixedPointDivisor>
      constexpr trunc_divide(const FixedPointDividend & lhs, const FixedPointDivisor & rhs) noexcept;
template <class FixedPoint>
    using trunc_reciprocal_result;
template <class FixedPoint>
    trunc_reciprocal_result<FixedPoint>
      constexpr trunc_reciprocal(const FixedPoint & fixed_point) noexcept;
template <class FixedPoint>
    using trunc_square_result;

template <class FixedPoint>
    trunc_square_result<FixedPoint>
      constexpr trunc_square(const FixedPoint & root) noexcept;
template <class FixedPoint>
    using trunc_sqrt_result;
```

```
  template <class FixedPoint>
      trunc_sqrt_result<FixedPoint>
        constexpr trunc_sqrt(const FixedPoint & square) noexcept;
  template <int Integer, class ReprType, int Exponent>
      constexpr fixed_point<ReprType, Exponent + Integer>
        trunc_shift_left(const fixed_point<ReprType, Exponent> & fp) noexcept;
  template <int Integer, class ReprType, int Exponent>
      constexpr fixed_point<ReprType, Exponent - Integer>
        trunc_shift_right(const fixed_point<ReprType, Exponent> & fp) noexcept;
  template <class FixedPoint, unsigned N = 2>
      using promote_add_result;

  template <class FixedPoint, class ... Tail>
      promote_add_result<FixedPoint, sizeof...(Tail) + 1>
        constexpr promote_add(const FixedPoint & addend1, const Tail & ... addend_tail);
  template <class Lhs, class Rhs = Lhs>
      using promote_subtract_result
  template <class Lhs, class Rhs>
      promote_subtract_result<Lhs, Rhs>
        constexpr promote_subtract(const Lhs & lhs, const Rhs & rhs) noexcept;
  template <class Lhs, class Rhs = Lhs>
      using promote_multiply_result;
  template <class Lhs, class Rhs>
      promote_multiply_result<Lhs, Rhs>
        constexpr promote_multiply(const Lhs & lhs, const Rhs & rhs) noexcept;
  template <class Lhs, class Rhs = Lhs>
      using promote_divide_result;
  template <class Lhs, class Rhs>
      promote_divide_result<Lhs, Rhs>
        constexpr promote_divide(const Lhs & lhs, const Rhs & rhs) noexcept;
  template <class FixedPoint>
      using promote_square_result;
  template <class FixedPoint>
      promote_square_result<FixedPoint>
        constexpr promote_square(const FixedPoint & root) noexcept;
}
```

Â `fixed_point<>Â` Class Template

```
template <class ReprType = int, int Exponent = 0>
class fixed_point
{
public:
  using repr_type = ReprType;

  constexpr static int exponent;
  constexpr static int digits;
  constexpr static int integer_digits;
  constexpr static int fractional_digits;

  fixed_point() noexcept;
  template <class S, typename std::enable_if<_impl::is_integral<S>::value, int>::type Dummy = 0>
    explicit constexpr fixed_point(S s) noexcept;
  template <class S, typename std::enable_if<std::is_floating_point<S>::value, int>::type Dummy = 0>
    explicit constexpr fixed_point(S s) noexcept;
  template <class FromReprType, int FromExponent>
    explicit constexpr fixed_point(const fixed_point<FromReprType, FromExponent> & rhs) noexcept;
  template <class S, typename std::enable_if<_impl::is_integral<S>::value, int>::type Dummy = 0>
    fixed_point & operator=(S s) noexcept;
  template <class S, typename std::enable_if<std::is_floating_point<S>::value, int>::type Dummy = 0>
    fixed_point & operator=(S s) noexcept;
  template <class FromReprType, int FromExponent>
    fixed_point & operator=(const fixed_point<FromReprType, FromExponent> & rhs) noexcept;

  template <class S, typename std::enable_if<_impl::is_integral<S>::value, int>::type Dummy = 0>
    explicit constexpr operator S() const noexcept;
  template <class S, typename std::enable_if<std::is_floating_point<S>::value, int>::type Dummy = 0>
    explicit constexpr operator S() const noexcept;
  explicit constexpr operator bool() const noexcept;

  template <class Rhs, typename std::enable_if<std::is_arithmetic<Rhs>::value, int>::type Dummy = 0>
    fixed_point &operator*=(const Rhs & rhs) noexcept;
  template <class Rhs, typename std::enable_if<std::is_arithmetic<Rhs>::value, int>::type Dummy = 0>
    fixed_point & operator/=(const Rhs & rhs) noexcept;

  constexpr repr_type data() const noexcept;
```

```
  static constexpr fixed_point from_data(repr_type repr) noexcept;
};
```

VI. Future Issues

Library Support

Because the aim is to provide an alternative to existing arithmetic types which are supported by the standard library, it is conceivable that a future proposal might specialize existing class templates and overload existing functions.

Possible candidates for overloading include the functions defined in \ and a templated specialization of `numeric_limits`. A new type trait, `is_fixed_point`, would also be useful.

While `fixed_point` is intended to provide drop-in replacements to existing built-ins, it may be preferable to deviate slightly from the behavior of certain standard functions. For example, overloads of functions from \ will be considerably less concise, efficient and versatile if they obey rules surrounding error cases. In particular, the guarantee of setting `errno` in the case of an error prevents a function from being defined as pure. This highlights a wider issue surrounding the adoption of the functional approach and compile-time computation that is beyond the scope of this document.

Alternatives to Built-in Integer Types

The reason that `ReprType` is restricted to built-in integer types is that a number of features require the use of a higher - or lower-capacity type. Supporting alias templates are defined to provide `fixed_point` with the means to invoke integer types of specific capacity and signedness at compile time.

There is no general purpose way of deducing a higher or lower-capacity type given a source type in the same manner as `make_signed` and `make_unsigned`. If there were, this might be adequate to allow alternative choices for `ReprType`.

Bounded Integers

The bounded::integer library [2] exemplifies the benefits of keeping track of ranges of values in arithmetic types at compile time.

To a limited extent, the `trunc_` functions defined here also keep track of - and modify - the limits of values. However, a combination of techniques is capable of producing superior results.

For instance, consider the following expression:

```
make_ufixed<2, 6> three(3);
auto n = trunc_square(trunc_square(three));
```

The type of `n` is `make_ufixed<8, 0>` but its value does not exceed 81. Hence, an integer bit has been wasted. It may be possible to track more accurate limits in the same manner as the bounded::integer library in order to improve the precision of types returned by `trunc_` functions. For this reason, the exact value of the exponents of these return types is not given.

Notes:

- Bounded::integer is already supported by fixed-point library, fp [3].
- A similar library is the boost constrained_value library [4].

Alternative Policies

The behavior of the types specialized from `fixed_point` represent one sub-set of all potentially desirable behaviors. Alternative characteristics include:

- different rounding strategies - other than truncation;
- overflow and underflow checks - possibly throwing exceptions;
- operator return type - adopting `trunc_` or `promote_` behavior;
- default-initialize to zero - currently uninitialized and
- saturation arithmetic - as opposed to modular arithmetic.

One way to extend `fixed_point` to cover these alternatives would be to add non-type template parameters containing bit flags or enumerated types. The default set of values would reflect `fixed_point` as it stands currently.

VII. Prior Art

Many examples of fixed-point support in C and C++ exist. While almost all of them aim for low run-time cost and expressive alternatives to raw integer manipulation, they vary greatly in detail and in terms of their interface.

One especially interesting dichotomy is between solutions which offer a discrete selection of fixed-point types and libraries which contain a continuous range of exponents through type parameterization.

N1169

One example of the former is found in proposal N1169 [5], the intent of which is to expose features found in certain embedded hardware. It introduces a succinct set of language-level fixed-point types and impose constraints on the number of integer or fractional digits each can possess.

As with all examples of discrete-type fixed-point support, the limited choice of exponents is a considerable restriction on the versatility and expressiveness of the API.

Nevertheless, it may be possible to harness performance gains provided by N1169 fixed-point types through explicit template specialization. This is likely to be a valuable proposition to potential users of the library who find themselves targeting platforms which support fixed-point arithmetic at the hardware level.

N3352

There are many other C++ libraries available which fall into the latter category of continuous-range fixed-point arithmetic [3] [6] [7]. In particular, an existing library proposal, N3352 [8], aims to achieve very similar goals through similar means and warrants closer comparison than N1169.

N3352 introduces four class templates covering the quadrant of signed versus unsigned and fractional versus integer numeric types. It is intended to replace built-in types in a wide variety of situations and accordingly, is highly compile-time configurable in terms of how rounding and overflow are handled. Parameters to these four class templates include the storage in bits and - for fractional types - the resolution.

The `fixed_point` class template could probably - with a few caveats - be generated using the two fractional types, `nonnegative` and `negatable`, replacing the `ReprType` parameter with the integer bit count of `ReprType`, specifying `fastest` for the rounding mode and specifying `undefined` as the overflow mode.

However, fixed_point more closely and concisely caters to the needs of users who already use integer types and simply desire a more concise, less error-prone form. It more closely follows the four design aims of the library and - it can be argued - more closely follows the spirit of the standard in its pursuit of zero-cost abstraction.

Some aspects of the design of the N3352 API which back up these conclusion are that:

- the result of arithmetic operations closely resemble the `trunc_` function templates and are potentially more costly at run-time;
- the nature of the range-specifying template parameters - through careful framing in mathematical terms - abstracts away valuable information regarding machine-critical type size information;
- the breaking up of duties amongst four separate class templates introduces four new concepts and incurs additional mental load for relatively little gain while further detaching the interface from vital machine-level details and
- the absence of the most negative number from signed types reduces the capacity of all types by one.

The added versatility that the N3352 API provides regarding rounding and overflow handling are of relatively low priority to users who already bear the scars of battles with raw integer types. Nevertheless, providing them as options to be turned on or off at compile time is an ideal way to leave the choice in the hands of the user.

Many high-performance applications - in which fixed-point is of potential value - favor run-time checks during development which are subsequently deactivated in production builds. The N3352 interface is highly conducive to this style of development. It is an aim of the fixed_point design to be similarly extensible in future revisions.

VIII. Acknowledgements

Subgroup: Guy Davidson, Michael Wong
Contributors: Ed Ainsley, Billy Baker, Lance Dyson, Marco Foco, ClÃ©ment GrÃ©goire, Nicolas Guillemot, Matt Kinzelman, JoÃ«l Lamotte, Sean Middleditch, Patrice Roy, Peter Schregle, Ryhor Spivak

IX. References

1. Why Integer Coordinates?, http://www.pathengine.com/Contents/Overview/FundamentalConcepts/WhyIntegerCoordinates/page.php
2. C++ bounded::integer library, http://doublewise.net/c++/bounded/
3. fp, C++14 Fixed Point Library, https://github.com/mizvekov/fp
4. Boost Constrained Value Libarary, http://rk.hekko.pl/constrained_value/
5. N1169, Extensions to support embedded processors, http://www.open-std.org/JTC1/SC22/WG14/www/docs/n1169.pdf
6. fpmath, Fixed Point Math Library, http://www.codeproject.com/Articles/37636/Fixed-Point-Class
7. Boost fixed_point (proposed), Fixed point integral and fractional types, https://github.com/viboes/fixed_point
8. N3352, C++ Binary Fixed-Point Arithmetic, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3352.html
9. fixed_point, Reference Implementation of P0037, https://github.com/johnmcfarlane/fixed_point

X. Appendix 1: Reference Implementation

An in-development implementation of the fixed_point class template and its essential supporting functions and types is available [9]. It includes a utility header containing such things as math and trigonometric functions and a partial `Â numeric_limitsÂ` specialization. Compile-time and run-time tests are included as well as benchmarking support. It is the source of examples and measurements cited here.

XI. Appendix 2: Performance

Despite a focus on usable interface and direct translation from integer-based fixed-point operations, there is an overwhelming expectation that the source code result in minimal instructions and clock cycles. A few preliminary numbers are presented to give a very early idea of how the API might perform.

Some notes:

- A few test functions were run, ranging from single arithmetic operations to basic geometric functions, performed against integer, floating-point and fixed-point types for comparison.
- Figures were taken from a single CPU, OS and compiler, namely:

```
Debian clang version 3.5.0-10 (tags/RELEASE_350/final) (based on LLVM 3.5.0)
Target: x86_64-pc-linux-gnu
Thread model: posix
```

- Fixed inputs were provided to each function, meaning that branch prediction rarely fails. Results may also not represent the full range of inputs.
- Details of the test harness used can be found in the source project mentioned in Appendix 1;
- Times are in nanoseconds;
- Code has not yet been optimized for performance.

Types

Where applicable various combinations of integer, floating-point and fixed-point types were tested with the following identifiers:

- `Â uint8_tÂ`, `Â int8_tÂ`, `Â uint16_tÂ`, `Â int16_tÂ`, `Â uint32_tÂ`, `Â int32_tÂ`, `Â uint64_tÂ` and `Â int64_tÂ` built-in integer types;
- `Â floatÂ`, `Â doubleÂ` and `Â long doubleÂ` built-in floating-point types;
- s3:4, u4:4, s7:8, u8:8, s15:16, u16:16, s31:32 and u32:32 format fixed-point types.

Basic Arithmetic

Plus, minus, multiplication and division were tested in isolation using a number of different numeric types with the following results:

name cpu_time
add(float) 1.78011
add(double) 1.73966
add(long double) 3.46011
add(u4_4) 1.87726
add(s3_4) 1.85051
add(u8_8) 1.85417
add(s7_8) 1.82057
add(u16_16) 1.94194
add(s15_16) 1.93463
add(u32_32) 1.94674
add(s31_32) 1.94446
add(int8_t) 2.14857
add(uint8_t) 2.12571
add(int16_t) 1.9936
add(uint16_t) 1.88229
add(int32_t) 1.82126
add(uint32_t) 1.76
add(int64_t) 1.76
add(uint64_t) 1.83223
sub(float) 1.96617
sub(double) 1.98491
sub(long double) 3.55474
sub(u4_4) 1.77006
sub(s3_4) 1.72983
sub(u8_8) 1.72983
sub(s7_8) 1.72983
sub(u16_16) 1.73966
sub(s15_16) 1.85051
sub(u32_32) 1.88229
sub(s31_32) 1.87063
sub(int8_t) 1.76
sub(uint8_t) 1.74994
sub(int16_t) 1.82126
sub(uint16_t) 1.83794
sub(int32_t) 1.89074
sub(uint32_t) 1.85417
sub(int64_t) 1.83703
sub(uint64_t) 2.04914
mul(float) 1.9376
mul(double) 1.93097
mul(long double) 102.446
mul(u4_4) 2.46583
mul(s3_4) 2.09189
mul(u8_8) 2.08
mul(s7_8) 2.18697
mul(u16_16) 2.12571
mul(s15_16) 2.10789
mul(u32_32) 2.10789
mul(s31_32) 2.10789
mul(int8_t) 1.76
mul(uint8_t) 1.78011
mul(int16_t) 1.8432

mul(uint16_t) 1.76914
mul(int32_t) 1.78011
mul(uint32_t) 2.19086
mul(int64_t) 1.7696
mul(uint64_t) 1.79017
div(float) 5.12
div(double) 7.64343
div(long double) 8.304
div(u4_4) 3.82171
div(s3_4) 3.82171
div(u8_8) 3.84
div(s7_8) 3.8
div(u16_16) 9.152
div(s15_16) 11.232
div(u32_32) 30.8434
div(s31_32) 34
div(int8_t) 3.82171
div(uint8_t) 3.82171
div(int16_t) 3.8
div(uint16_t) 3.82171
div(int32_t) 3.82171
div(uint32_t) 3.81806
div(int64_t) 10.2286
div(uint64_t) 8.304

Among the slowest types are `Â long doubleÂ`. It is likely that they are emulated in software. The next slowest operations are fixed-point multiply and divide operations - especially with 64-bit types. This is because values need to be promoted temporarily to double-width types. This is a known fixed-point technique which inevitably experiences slowdown where a 128-bit type is required on a 64-bit system.

Here is a section of the disassembly of the s15:16 multiply call:

```
30:   mov    %r14,%rax
      mov    %r15,%rax
      movslq -0x28(%rbp),%rax
      movslq -0x30(%rbp),%rcx
      imul   %rax,%rcx
      shr    $0x10,%rcx
      mov    %ecx,-0x38(%rbp)
      mov    %r12,%rax
4c:   movzbl (%rbx),%eax
      cmp    $0x1,%eax
    â jne    68
54:   mov    0x8(%rbx),%rax
      lea    0x1(%rax),%rcx
      mov    %rcx,0x8(%rbx)
      cmp    0x38(%rbx),%rax
    â jb     30
```

The two 32-bit numbers are multiplied together and the result shifted down - much as it would if raw `Â intÂ` values were used. The efficiency of this operation varies with the exponent. An exponent of zero should mean no shift at all.

3-Dimensional Magnitude Squared

A fast `Â sqrtÂ` implementation has not yet been tested with `Â fixed_pointÂ`. (The naive implementation takes over 300ns.) For this reason, a magnitude-squared function is measured, combining multiply and add operations:

```
template <typename FP>
constexpr FP magnitude_squared(const FP & x, const FP & y, const FP & z)
{
    return x * x + y * y + z * z;
}
```

Only real number formats are tested:

float 2.42606
double 2.08
long double 4.5056
s3_4 2.768
s7_8 2.77577
s15_16 2.752
s31_32 4.10331

Again, the size of the type seems to have the largest impact.

Circle Intersection

A similar operation includes a comparison and branch:

```
template <typename Real>
bool circle_intersect_generic(Real x1, Real y1, Real r1, Real x2, Real y2, Real r2)
{
    auto x_diff = x2 - x1;
    auto y_diff = y2 - y1;
    auto distance_squared = x_diff * x_diff + y_diff * y_diff;

    auto touch_distance = r1 + r2;
    auto touch_distance_squared = touch_distance * touch_distance;

    return distance_squared <= touch_distance_squared;
}
```

float 3.46011
double 3.48
long double 6.4
s3_4 3.88
s7_8 4.5312
s15_16 3.82171
s31_32 5.92

Again, fixed-point and native performance are comparable.